K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

DATA STRUCTURES

ASSIGNMENT – 2

TITLE : Linear Data Structures Toolkit (LDST)

SUMBITTED TO :

MRS. NEETU CHAUHAN

SUMBITTED BY :

NAME : SANIA KAMBOJ

PROGRAM :

BTECH CSE (AI & ML)

ROLL NO : 2501730319

SECTION : A

SEMESTER : 2nd

BATCH : 2025-2029

#Dynamic Array Simulation :

Amortized Time Complexity of append() :

A **dynamic array** (like Python list) automatically increases its size when it becomes full. The append() operation is used to add a new element at the end of the array.

Case 1: When array has space (Normal Case)

- If the array is **not full**, the new element is directly placed at the next index.
- No shifting or copying is needed.

Time taken = **O(1)** (constant time)

Example:
Array: `[1, 2, 3, _ , _]`
Append `4` → `[1, 2, 3, 4, _]`

Case 2: When array is full (Resizing Case)

- If the array is **full**, we cannot add a new element directly.
- So we:
    1. Create a new array of **double capacity**
    2. Copy all existing elements to the new array
    3. Insert the new element

Time taken = **O(n)** (because copying n elements)

Example:
Old array: `[1, 2, 3, 4]` (full)
New array: `[1, 2, 3, 4, _, _, _, _]`

Why resizing does NOT make it slow overall?

Resizing looks expensive (**O(n)**), but:

- It **does not happen every time**
- It happens only when the array becomes full

- After resizing, many future appends will again be **O(1)**

Amortized Analysis

Instead of looking at a **single operation**, we look at **many operations together**.

Let's say we perform **n appends**:

- Most operations → **O(1)**
- Few operations (resizing) → **O(n)**

If we spread (average) the total cost over all operations:

Average cost per operation = **O(1)**

Intuition (Very Easy Understanding)

Think of it like this:

- You pay a **small cost most of the time**
- Occasionally, you pay a **big cost (resizing)**
- But that big cost is shared across many operations

So overall cost per append remains small

#Time Complexity of Stack and Queue Operations

Stack Operations (LIFO) :

A Stack **follows** Last In, First Out (LIFO) **principle. The last element inserted is removed first.**

1. Push(x) → O(1)

- Push means adding an element at the top of the stack.
- In linked list implementation, we insert at the head/top directly.
- No traversal is needed.

Therefore, push takes **constant time O(1)**.

2. Pop() → O(1)

- Pop means removing the top element from the stack.
- We remove the head/top node directly.

Therefore, pop also takes **O(1)** time.

3. Peek() → O(1)

- Peek means viewing the top element without removing it.
- We simply return the top node data.

Therefore, peek takes **O(1)** time.

Queue Operations (FIFO)

A **Queue** follows **First In, First Out (FIFO)** principle.
The first inserted element is removed first.

1. Enqueue(x) → O(1)

- Enqueue means inserting an element at the rear of the queue.
- Using head and tail pointers, we add directly at tail.

Therefore, enqueue takes **O(1)** time.

2. Dequeue() → O(1)

- Dequeue means removing the front element.
- We remove directly from head/front.

Therefore, dequeue takes **O(1)** time.

3. Front() → O(1)

- Front means viewing the first element without removing it.
- We directly access head node data.

Therefore, front takes **O(1)** time.

Summary Table

Data Structure	Operation	Time Complexity	Reason
Stack	Push	O(1)	Insert at top
Stack	Pop	O(1)	Remove top
Stack	Peek	O(1)	Access top
Queue	Enqueue	O(1)	Insert at rear
Queue	Dequeue	O(1)	Remove front
Queue	Front	O(1)	Access front